

Assignment No 1 – 10 mark Questions

Breadth First Search (BFS)

Breadth First Search is an uninformed search technique that explores states level by level. It uses a FIFO queue. The initial state is inserted into the queue. The first state is removed, its goal condition is checked, and its unvisited successors are added to the queue. This process continues until the goal is found or the queue becomes empty.

BFS Step-by-Step Algorithm

1. Start with the initial state.
2. Insert the initial state into the queue.
3. Remove the first state from the queue.
4. Check whether the removed state is the goal state.
5. If it is the goal, stop and report the solution.
6. Otherwise, generate its successor states.
7. Add unvisited successor states to the queue.
8. Mark the processed state as visited.
9. Repeat the process until the goal is reached or the queue is empty.

Q1: Write a program to implement Breadth First Search (BFS) for the Monkey Banana Problem.

Objective: To implement BFS for solving the Monkey Banana Problem.

States used in this simplified program:

- Monkey at Door
- Monkey at Box
- Box Under Banana
- Monkey Climbs Box
- Monkey Gets Banana

Algorithm:

1. Initialize a FIFO queue with the initial state 'Monkey at Door'.
2. Create a visited set.
3. Remove a state from the front of the queue.
4. If it has already been visited, skip it.
5. Print and mark the state as visited.
6. Generate the next state according to the problem sequence.
7. When 'Monkey Gets Banana' is reached, display the goal message and stop.

Python Program:

```

from collections import deque
# BFS for Monkey Banana Problem
queue = deque(["Monkey at Door"])
visited = set()
while queue:
    state = queue.popleft()

    if state in visited:
        continue
    visited.add(state)
    print(state)

    if state == "Monkey at Door":
        queue.append("Monkey at Box")
    elif state == "Monkey at Box":
        queue.append("Box Under Banana")
    elif state == "Box Under Banana":
        queue.append("Monkey Climbs Box")
    elif state == "Monkey Climbs Box":
        queue.append("Monkey Gets Banana")
    elif state == "Monkey Gets Banana":
        print("Goal: Banana Obtained!")
        break

```

Expected Output:

```

Monkey at Door
Monkey at Box
Box Under Banana
Monkey Climbs Box
Monkey Gets Banana
Goal: Banana Obtained!

```

Q2: Write a program to solve the Water Jug Problem using BFS.

Objective: To obtain exactly 2 litres using a 4-litre jug and a 3-litre jug using BFS.

State representation: (x, y), where x is the amount in the 4-litre jug and y is the amount in the 3-litre jug.

Initial state: (0, 0)

Goal state: x = 2 or y = 2

Possible operations: Fill either jug, empty either jug, or transfer water from one jug to the other until the source is empty or the destination is full.

Python Program:

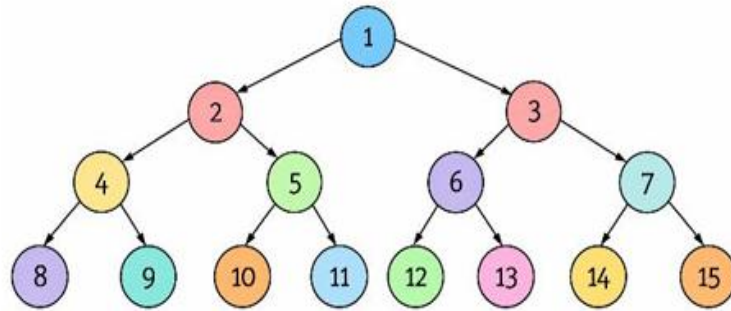
```
from collections import deque
def water_jug():
    visited = set()
    queue = deque([(0, 0)])

    while queue:
        x, y = queue.popleft()
        if (x, y) in visited:
            continue
        visited.add((x, y))

        if x == 2 or y == 2:
            print("Solution:", (x, y))
            return
        states = [
            (4, y), # Fill 4-litre jug
            (x, 3), # Fill 3-litre jug
            (0, y), # Empty 4-litre jug
            (x, 0), # Empty 3-litre jug
            # Pour 4-litre jug into 3-litre jug
            (max(0, x - (3 - y)), min(3, x + y)),
            # Pour 3-litre jug into 4-litre jug
            (min(4, x + y), max(0, y - (4 - x)))
        ]
        for state in states:
            if state not in visited:
                queue.append(state)
    water_jug()
```

Expected Output: Solution: (2, 3)

Q3: Write a program to implement BFS traversal for a graph.



Objective: To traverse a graph level by level using BFS.

Algorithm:

1. Represent the graph using an adjacency list.
2. Insert the starting vertex into the queue.
3. Mark the starting vertex as visited.
4. Remove the first vertex from the queue and print it.
5. Insert all unvisited adjacent vertices into the queue and mark them visited.
6. Repeat until the queue is empty.

Python Program:

```

from collections import deque
# Graph represented using adjacency list
graph = {
    1: [2, 3],
    2: [4, 5],
    3: [6, 7],
    4: [8, 9],
    5: [10, 11],
    6: [12, 13],
    7: [14, 15],
    8: [],
    9: [],
    10: [],
    11: [],
    12: [],
    13: [],
    14: [],
    15: []
}

```

```

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)

```

```
print("BFS Traversal:")
```

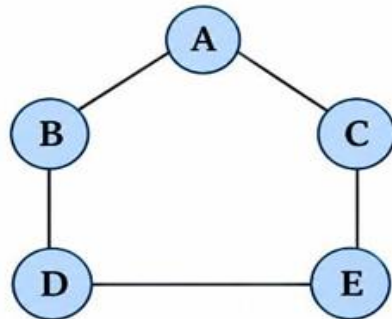
```
while queue:
    node = queue.popleft()
    print(node, end=" ")
    for neighbour in graph[node]:
        if neighbour not in visited:
            visited.add(neighbour)
            queue.append(neighbour)
```

```
# Start BFS from node 1
```

```
bfs(graph, 1)
```

Q4 : Write a program to solve a state-space search problem using BFS.

Start Vertex = A



Example: Find a path from a start state to a goal state in a small state-space graph.

Algorithm:

1. Define the states and their connections.
2. Start BFS from the initial state.
3. Store the path along with each state.
4. When a state is reached, check whether it is the goal.
5. If the goal is reached, display the path.

Python Program:

```
from collections import deque
```

```
# State-space graph
```

```
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'E'],
    'D': ['B', 'E'],
    'E': ['C', 'D']
}
```

```
def bfs(start):
    visited = set()
```

```

queue = deque()

# Add start vertex
queue.append(start)
visited.add(start)

print("BFS Traversal:")

while queue:
    vertex = queue.popleft()
    print(vertex, end=" ")

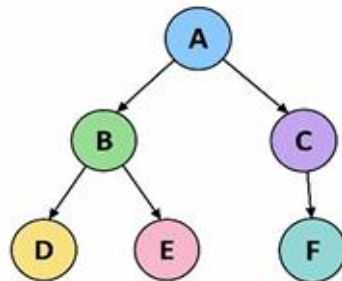
    # Visit all adjacent states
    for neighbour in graph[vertex]:
        if neighbour not in visited:
            visited.add(neighbour)
            queue.append(neighbour)

# Start BFS from A
bfs('A')

```

Output : BFS Traversal: A B C D E

Q5: Write a program implement Iterative Deepening Search (IDS).



Python Program :

Iterative Deepening Search (IDS)

```

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

def depth_limited_search(node, depth):
    print(node, end=" ")

```

```

if depth == 0:
    return

for child in graph[node]:
    depth_limited_search(child, depth - 1)

def iterative_deepening_search(start, max_depth):
    for depth in range(max_depth + 1):
        print("\nDepth", depth, ":", end=" ")
        depth_limited_search(start, depth)

# Start vertex
start = 'A'

# Maximum depth
max_depth = 2

print("Iterative Deepening Search:")
iterative_deepening_search(start, max_depth)

```

Output :

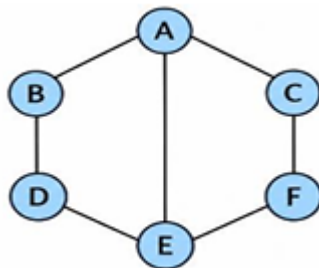
Iterative Deepening Search:

Depth 0 : A

Depth 1 : A B C

Depth 2 : A B D E C F

Q6: Write a program to implement DFS traversal for a graph.



Objective: To traverse a graph using Depth First Search.

Python Program:

DFS Traversal of a Graph

```

graph = {
    'A': ['B', 'C', 'E'],
    'B': ['A', 'D'],

```

```

'C': ['A', 'F'],
'D': ['B', 'E'],
'E': ['A', 'D', 'F'],
'F': ['C', 'E']
}

visited = set()

def dfs(vertex):
    # Mark the vertex as visited
    visited.add(vertex)
    print(vertex, end=" ")

    # Visit all unvisited adjacent vertices
    for neighbour in graph[vertex]:
        if neighbour not in visited:
            dfs(neighbour)

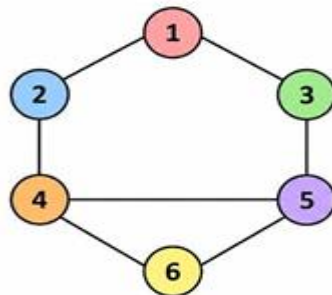
# Start DFS from A
print("DFS Traversal:")
dfs('A')

```

Output: DFS Traversal: A B D E F C

Q7 : Write a program to solve a state-space search problem using DFS.

Start vertex - 1



Start Vertex = 1

For the given state-space graph, the connections are:

- 1 → 2, 3

- $2 \rightarrow 4$
- $3 \rightarrow 5$
- $4 \rightarrow 5, 6$
- $5 \rightarrow 6$

Python Program:

DFS for State-Space Search

```
graph = {
    1: [2, 3],
    2: [1, 4],
    3: [1, 5],
    4: [2, 5, 6],
    5: [3, 4, 6],
    6: [4, 5]
}

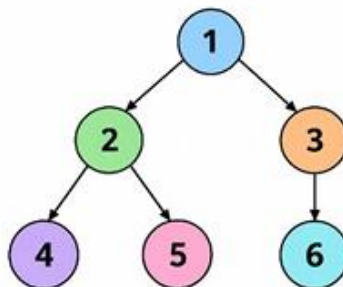
visited = set()

def dfs(vertex):
    # Mark the vertex as visited
    visited.add(vertex)
    print(vertex, end=" ")

    # Visit unvisited adjacent vertices
    for neighbour in graph[vertex]:
        if neighbour not in visited:
            dfs(neighbour)

# Start DFS from vertex 1
print("DFS State-Space Search:")
dfs(1)
```

Q8 : Write a program to implement Depth Limited Search (DLS).



Assume Maximum Depth = 2.

Python Program:

Depth-Limited Search (DLS)

```
graph = {
    1: [2, 3],
    2: [4, 5],
    3: [6],
    4: [],
    5: [],
    6: []
}

def dls(node, depth):
    print(node, end=" ")

    # Stop when depth limit is reached
    if depth == 0:
        return

    # Visit child nodes
    for child in graph[node]:
        dls(child, depth - 1)

# Start vertex and depth limit
start = 1
max_depth = 2

print("Depth-Limited Search:")
dls(start, max_depth)
```

Output : Depth-Limited Search: 1 2 4 5 3 6

Q 9: Write a program to solve the Tower of Hanoi Problem using state-space representation.

For the Tower of Hanoi problem:

- A, B, C = three pegs
- Initial State: (A, A, A) — all disks are on peg A
- Goal State: (C, C, C) — all disks are moved to peg C
- Each state represents the position of the disks.
- A move changes the position of one disk while following the Tower of Hanoi rules.

For example, with 3 disks: Initial State: (A, A, A)
Goal State: (C, C, C)

Python Program:

Tower of Hanoi using State-Space Representation

```
def tower_of_hanoi(n, source, auxiliary, destination, state):

    # Base condition
    if n == 0:
        return

    # Move n-1 disks from source to auxiliary
    tower_of_hanoi(n - 1, source, destination, auxiliary, state)

    # Move the largest disk to destination
    print("Move disk", n, "from", source, "to", destination)

    # Update state
    state[n - 1] = destination
    print("State:", tuple(state))

    # Move n-1 disks from auxiliary to destination
    tower_of_hanoi(n - 1, auxiliary, source, destination, state)

# Number of disks
n = 3

# Initial state: all disks are on A
state = ['A'] * n

print("Initial State:", tuple(state))
print("\nSteps:")

tower_of_hanoi(n, 'A', 'B', 'C', state)
print("\nGoal State:", tuple(state))
```

Output :

Initial State: ('A', 'A', 'A')

Steps:

Move disk 1 from A to C

State: ('C', 'A', 'A')

Move disk 2 from A to B

State: ('C', 'B', 'A')

Move disk 1 from C to B

State: ('B', 'B', 'A')

Move disk 3 from A to C

State: ('B', 'B', 'C')

Move disk 1 from B to A

State: ('A', 'B', 'C')

Move disk 2 from B to C

State: ('A', 'C', 'C')

Move disk 1 from A to C

State: ('C', 'C', 'C')

Goal State: ('C', 'C', 'C')

State-Space Sequence

For 3 disks, the state changes from:

(A,A,A)

↓

(C,A,A)

↓

(B,B,A)

↓

(B,B,A)

↓

(B,B,C)

↓

(A,B,C)

↓

(A,C,C)

↓

(C,C,C)

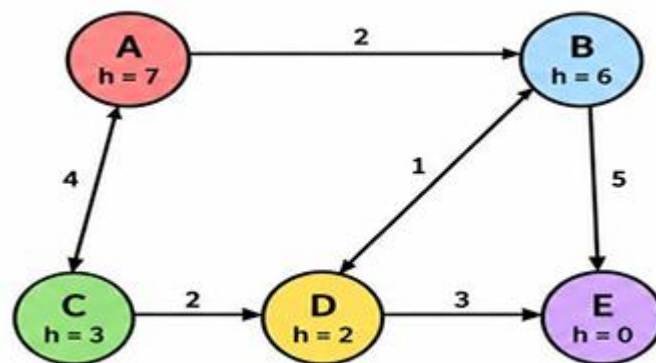
Minimum number of moves = $2^3 - 1 = 7$

Assignment No 2 – 10 mark Questions

A* Search

A* is an intelligent search algorithm used to find a shortest or best path. It uses both the actual cost from the start node and the estimated cost to the goal. Evaluation function: $f(n) = g(n) + h(n)$ filecite
Here, $g(n)$ is the actual cost from the start to node n , $h(n)$ is the estimated cost from n to the goal, and $f(n)$ is the total estimated cost.

Q1 : Write a program to implement the A* Search Algorithm to find the shortest path between a source node and a destination node.



Assume:

- Source Node: A
- Destination Node: F
- Cost of each edge: 1
- Heuristic values (h): estimated cost from each node to F

Node	h(n)
A	2
B	3
C	1
D	4
E	4
F	0

A* uses: $f(n) = g(n) + h(n)$

where:

- $g(n)$ = actual cost from source to current node
- $h(n)$ = estimated cost from current node to destination
- $f(n)$ = total estimated cost

Python Program

```
import heapq
```

Graph with edge costs

```
graph = {
    'A': [('B', 1), ('C', 1)],
    'B': [('D', 1), ('E', 1)],
    'C': [('F', 1)],
    'D': [],
    'E': [],
    'F': []
}
```

Heuristic values: estimated cost to reach F

```
heuristic = {
    'A': 2,
    'B': 3,
    'C': 1,
    'D': 4,
    'E': 4,
    'F': 0
}
```

```
def a_star(source, destination):
```

```
    # Priority queue: (f, g, node, path)
```

```
    open_list = []
```

```
    heapq.heappush(open_list, (heuristic[source], 0, source, [source]))
```

```
    visited = set()
```

```
    while open_list:
```

```
        f, g, current, path = heapq.heappop(open_list)
```

```
        if current in visited:
```

```
            continue
```

```
        visited.add(current)
```

```
        # Destination reached
```

```
        if current == destination:
```

```
            return path, g
```

```
        # Explore neighbouring nodes
```

```
        for neighbour, cost in graph[current]:
```

```
            if neighbour not in visited:
```

```
                new_g = g + cost
```

```
                new_f = new_g + heuristic[neighbour]
```

```
                heapq.heappush(
```

```

        open_list,
        (new_f, new_g, neighbour, path + [neighbour])
    )

    return None, None

# Source and destination
source = 'A'
destination = 'F'
path, cost = a_star(source, destination)
print("Source Node:", source)
print("Destination Node:", destination)
print("Shortest Path:", " -> ".join(path))
print("Path Cost:", cost)

```

Output

Source Node: A
 Destination Node: F
 Shortest Path: A -> C -> F
 Path Cost: 2

Working of A***Initially:**

A
 From A, we have B and C.

For B:

$g(B) = 1$
 $h(B) = 3$
 $f(B) = 1 + 3 = 4$

For C:

$g(C) = 1$
 $h(C) = 1$
 $f(C) = 1 + 1 = 2$
 Since C has the smaller $f(n)$ value, A* selects C.

Then:

$C \rightarrow F$

For F:

$g(F) = 2$
 $h(F) = 0$
 $f(F) = 2 + 0 = 2$
 Therefore, the destination F is reached.

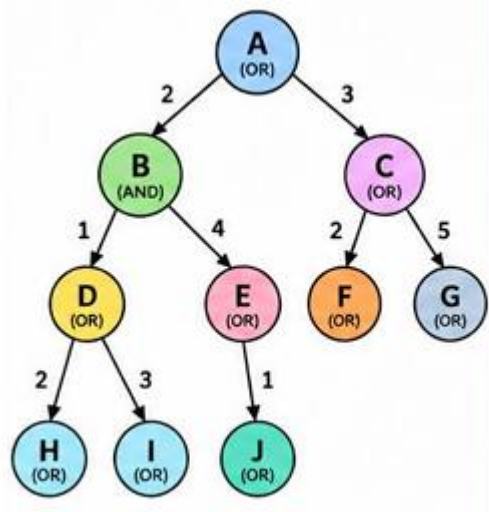
Final Answer Shortest Path: $A \rightarrow C \rightarrow F$

Total Cost = 2

AO* Search

AO* is an informed search algorithm for AND-OR graphs. At an OR node, one child is sufficient; at an AND node, all required child nodes must be solved. Heuristic values and cost evaluations are used to refine the solution graph.

Q2 : Write the program to implement the AO* Algorithm



For the given AND-OR graph:

- **A** is an **OR** node → choose either B or C.
- **B** is an **AND** node → both D and E must be solved.
- **C** is an **OR** node → choose either F or G.
- **D** is an **OR** node → choose H or I.
- **E** is an **OR** node → choose J.

Python Program :

AO* Algorithm for AND-OR Graph

```

graph = {
    'A': {
        'type': 'OR',
        'children': [('B', 2), ('C', 3)]
    },

    'B': {
        'type': 'AND',
        'children': [('D', 1), ('E', 4)]
    },

    'C': {

```

```
'type': 'OR',
'children': [('F', 2), ('G', 5)]
},
```

```
'D': {
'type': 'OR',
'children': [('H', 2), ('I', 3)]
},
```

```
'E': {
'type': 'OR',
'children': [('J', 1)]
},
```

```
# Goal nodes
```

```
'F': {'type': 'GOAL', 'children': []},
'G': {'type': 'GOAL', 'children': []},
'H': {'type': 'GOAL', 'children': []},
'I': {'type': 'GOAL', 'children': []},
'J': {'type': 'GOAL', 'children': []}
}
```

```
def ao_star(node):
```

```
    # If node is a goal node
```

```
    if graph[node]['type'] == 'GOAL':
```

```
        return 0, [node]
```

```
    children = graph[node]['children']
```

```
    if graph[node]['type'] == 'OR':
```

```
        # Choose the child having minimum cost
```

```
        best_cost = float('inf')
```

```
        best_path = None
```

```
        for child, edge_cost in children:
```

```
            child_cost, child_path = ao_star(child)
```

```
            total_cost = edge_cost + child_cost
```

```
            if total_cost < best_cost:
```

```
                best_cost = total_cost
```

```
                best_path = [node] + child_path
```

```
        return best_cost, best_path
```

```
    elif graph[node]['type'] == 'AND':
```

```
        # All children must be solved
```

```

total_cost = 0
path = [node]

for child, edge_cost in children:
    child_cost, child_path = ao_star(child)

    total_cost += edge_cost + child_cost
    path += child_path
return total_cost, path

# Start AO* from node A
cost, solution = ao_star('A')

print("AO* Solution:")
print(" -> ".join(solution))
print("Total Cost:", cost)

```

Output**AO* Solution:** A -> C -> F**Total Cost:** 5

Q3: Write a program to demonstrate the Hill Climbing Algorithm for finding the maximum value of a simple objective function. ($F(x) = -x^2 + 4x$)

Python Program:

```

# Hill Climbing Algorithm
# Objective Function:  $F(x) = -x^2 + 4x$ 

```

```

def f(x):
    return -x**2 + 4*x

def hill_climbing(start, step=1):
    current = start

    while True:
        current_value = f(current)

        # Generate neighbouring values
        left = current - step
        right = current + step

        # Select the better neighbour
        if f(left) > current_value:
            current = left
        elif f(right) > current_value:

```

```

        current = right
    else:
        break

    return current, f(current)

# Starting point
start = 0

# Apply Hill Climbing
x, maximum = hill_climbing(start)

print("Starting Point:", start)
print("Maximum value occurs at x =", x)
print("Maximum value of F(x) =", maximum)

```

Output

Starting Point: 0
 Maximum value occurs at x = 2
 Maximum value of F(x) = 4

Step-by-step

Starting from x = 0:

Current x	F(x)	Better move
0	0	Move to 1
1	3	Move to 2
2	4	Stop

At x = 2:

$$F(2) = -(2)^2 + 4(2) = -4 + 8 = 4$$

The neighboring points x = 1 and x = 3

both give:

$$F(1) = 3, F(3) = 3$$

Both are lower than $F(2) = 4$, so the algorithm stops.

Result

Maximum value = 4

At x = 2

$$y = -x^2 + 4x, (2, 4)$$

Hill Climbing: Start at a point → evaluate neighboring points → move toward a better value → stop when no neighbor is better.

Q4 : Write a program to solve the 4-Puzzle Problem using the A* Search Algorithm.

A 4-Puzzle consists of **4 tiles and one blank space** arranged in a 2×2 grid.

For this program: Initial State:

1 0
3 2

Here 0 represents the blank space.

Goal State: 1 2
3 0

A* uses: $f(n)=g(n)+h(n)$ $f(n) = g(n) + h(n)$

where:

- $g(n)$ = cost from the initial state to the current state
- $h(n)$ = Manhattan distance from the current state to the goal
- $f(n)$ = total estimated cost

Python Program:

```
import heapq
```

```
# Initial and goal states
```

```
start = (1, 0, 3, 2)
```

```
goal = (1, 2, 3, 0)
```

```
# Manhattan Distance Heuristic
```

```
def heuristic(state):
```

```
    distance = 0
```

```
    for i in range(4):
```

```
        if state[i] == 0:
```

```
            continue
```

```
        # Current position
```

```
        row1 = i // 2
```

```
        col1 = i % 2
```

```
        # Goal position
```

```
        goal_index = goal.index(state[i])
```

```
        row2 = goal_index // 2
```

```
        col2 = goal_index % 2
```

```
        distance += abs(row1 - row2) + abs(col1 - col2)
```

```
    return distance
```

```
# Generate possible next states
```

```
def get_neighbors(state):
```

```
    neighbors = []
```

```
    blank = state.index(0)
```

```

row = blank // 2
col = blank % 2
moves = [
    (-1, 0), # Up
    (1, 0),  # Down
    (0, -1), # Left
    (0, 1)   # Right
]

for dr, dc in moves:
    new_row = row + dr
    new_col = col + dc

    if 0 <= new_row < 2 and 0 <= new_col < 2:

        new_blank = new_row * 2 + new_col
        new_state = list(state)
        # Swap blank with tile
        new_state[blank], new_state[new_blank] = \
            new_state[new_blank], new_state[blank]

        neighbors.append(tuple(new_state))

return neighbors

# A* Search Algorithm
def a_star(start, goal):

    # Priority queue: (f, g, state, path)
    open_list = []
    heapq.heappush(
        open_list,
        (heuristic(start), 0, start, [start])
    )
    visited = set()
    while open_list:

        f, g, current, path = heapq.heappop(open_list)

        if current in visited:
            continue
        visited.add(current)

        # Goal reached
        if current == goal:
            return path

```

```

# Generate next states
for neighbor in get_neighbors(current):

    if neighbor not in visited:
        new_g = g + 1
        new_f = new_g + heuristic(neighbor)
        heapq.heappush(
            open_list,
            (new_f, new_g, neighbor, path + [neighbor])
        )

    return None

# Solve the puzzle
solution = a_star(start, goal)

print("Initial State:")
print("1 0")
print("3 2")

print("\nGoal State:")
print("1 2")
print("3 0")

print("\nSolution Path:")

for state in solution:
    print()
    print(state[0], state[1])
    print(state[2], state[3])

```

Output

Initial State:

1 0

3 2

Goal State:

1 2

3 0

Solution Path:

1 0

3 2

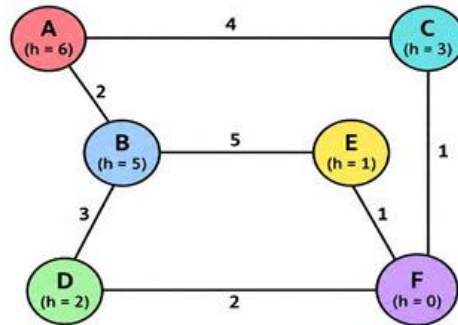
1 2

3 0

Best First Search

Best First Search is a heuristic search technique that selects the node with the greatest potential to lead toward the solution according to its heuristic value. The OPEN list works like a priority queue, and the node with the lowest heuristic value is selected.

Q5 : Write a program to implement the Best First Search Algorithm using heuristic values to find a path from a start node to a goal node.



- **Start Node: A**
- **Goal Node: F**
- **Heuristic values:**
 - **A = 6**
 - **B = 5**
 - **C = 3**
 - **D = 2**
 - **E = 1**
 - **F = 0**

Best First Search selects the node having the lowest heuristic value $h(n)$.

Python Program:

```
import heapq
```

```
# Graph with edge costs
```

```
graph = {
    'A': [('B', 2), ('C', 4)],
    'B': [('A', 2), ('D', 3), ('E', 5)],
    'C': [('A', 4), ('F', 1)],
    'D': [('B', 3), ('F', 2)],
    'E': [('B', 5), ('F', 1)],
    'F': [('C', 1), ('D', 2), ('E', 1)]
}
```

```
# Heuristic values
```

```
heuristic = {
    'A': 6,
    'B': 5,
```

```
'C': 3,
'D': 2,
'E': 1,
'F': 0
}
```

```
def best_first_search(start, goal):
    # Priority queue: (heuristic, node, path)
    open_list = []

    heapq.heappush(open_list, (heuristic[start], start, [start]))

    visited = set()

    while open_list:
        h, current, path = heapq.heappop(open_list)
        if current in visited:
            continue

        visited.add(current)
        print("Visiting:", current, "h =", heuristic[current])

        # Goal test
        if current == goal:
            return path

        # Add unvisited neighbours
        for neighbour, cost in graph[current]:
            if neighbour not in visited:
                heapq.heappush(
                    open_list,
                    (heuristic[neighbour], neighbour, path + [neighbour])
                )

    return None

# Start and goal
start = 'A'
goal = 'F'

path = best_first_search(start, goal)
print("\nBest First Search Path:")
print(" -> ".join(path))
```

Output

Visiting: A h = 6

Visiting: C $h = 3$

Visiting: F $h = 0$

Best First Search Path: A $\rightarrow$ C $\rightarrow$ F